

An Efficient Framework for Collusion Resistant Multiparty FHE

Robin Jadoul^{1*} , Barry van Leeuwen² , and Oliver Zajonc² 

¹ 3MI Labs, Leuven, Belgium.

`robin.jadoul@3milabs.tech`

² COSIC, ESAT, KU Leuven, Leuven, Belgium.

`{barry.vanleeuwen, oliver.zajonc}@kuleuven.be`

Abstract. Multiparty fully homomorphic encryption (MPFHE) is a generalization of (multi-key) fully homomorphic encryption ((MK)FHE) that lives on the cusp between multiparty computation (MPC) and FHE, enabling a computation over encrypted data using multiple keys. However, contrary to MKFHE, it seeks to reduce the noise inflation based on the number of parties by allowing the parties to first compute shared data in MPC before executing the computation in FHE. However, many works use specific constructions [CL25,KTSJ24,MWA⁺23] reliant on the behaviour of a particular scheme and do not consider security against colluding servers and parties [PS24,OT25,MTBH21]. In this work we present a new framework that is secure against collusion between servers and parties and allows the use of a broad spectrum of MPC and FHE protocols, while eliminating the noise inflation based on the participating number of parties. We then show a case study of this using the FINAL scheme and show that we reduce the required key material by 40-99.9% compared to the MKFHE FINAL scheme, FINALLY, 8-71% compared to the static AKÖ scheme, and 65-70% compared to the Park-Rovira scheme. Moreover, we reduce the bootstrapping time for the AKÖ, Park-Rovira, and KMS schemes by 75-99.7%.

Keywords: Multiparty Computation · Fully Homomorphic Encryption · Multiparty FHE · LWE · NTRU · Key Switching

* Part of this work was performed while the author was at COSIC, KU Leuven.

Table of Contents

1	Introduction	3
1.1	Our Contributions	4
1.2	Related Work	5
2	Preliminaries	5
2.1	Notation	5
2.2	Distributions	6
2.3	Gadget Decomposition	6
2.4	Single-Key NGS	7
2.5	Multi-key LWE	7
2.6	Key Switching	8
2.7	Modulus Switching	9
2.8	Multiparty Computation	10
3	A Framework for LWE-based Multiparty FHE Schemes	11
3.1	Single-Key FHE: Security versus Efficiency	11
3.2	Generalized Multiparty FHE	12
3.3	The Framework in Practice	13
3.4	Practical Considerations for the Choice of MPC and FHE Schemes	15
4	A Case Study: Multiparty FINAL	16
4.1	Key Generation	16
4.2	Bootstrapping	17
4.3	Heuristics	20
4.4	Decryption	21
4.5	Key Compression	21
5	Implementation	22

1 Introduction

Computing on Encrypted Data (COED) is a branch of privacy enhancing technologies (PET), with two main approaches: multiparty computation (MPC) and fully homomorphic encryption (FHE). Both enable parties to jointly compute on private inputs, but through very different mechanisms.

In most cases, MPC distributes the secrets in a safe manner (such as secret sharing), allowing each of the parties to perform computation on their own machine before communicating the results with the other parties in the protocol. These systems allow obtaining high security guarantees at the cost of high communication and computation complexity. This leads to two major drawbacks: Each of the parties has to be present throughout the execution of the protocol and require a computationally powerful machine.

In contrast, FHE outsources computation to a server without revealing inputs or outputs, reducing overhead and letting parties go offline. However, this does come at the cost of much lower security guarantees. First and foremost, due to the nature of homomorphic encryption, the server needs to be considered semi-honest.³ Another common assumption [LTV12,BCG⁺17,PS24,OT25] is that in a setting with at least two users and the server, the server and a user can not collude. If there is a single key, then the server could receive the encryption of the secret of the honest party and the adversary - knowing the key as there is a dishonest party - could decrypt that message, learning the honest party's private input. A standard solution to this is the introduction of (unique) keys for each of the parties. This augmentation, called Multi-key FHE [MW16,AJMM20,PVZ24], provides a higher security guarantee, but it also introduces a significant constraint.

Most FHE schemes achieve the combination of functionality and security by introducing noise, which prevents an adversary from learning anything about the message in a ciphertext. To remove the noise, the decryption step usually has a rounding function which ensures the correct retrieval of the underlying message dependent on the amount of noise present in the ciphertext. With every operation the noise grows: linearly for addition, but exponentially for multiplication. It will come as no surprise that after a number of operations the noise can grow so large that the rounding function cannot reliably round back to the underlying message. To accommodate the size of arbitrary computations a new protocol depending on the scheme was introduced called *bootstrapping*. While this allowed for arbitrary computations to be executed, the bootstrapping protocol is often very computationally expensive when compared to the arithmetic operations. One of the largest constraints with Multi-key FHE schemes is that the noise grows significantly faster and is often related to the number of parties that participate in the protocol. As a result, these multi-key protocols often require much larger parameter sets than their single-key counterparts.

Multiparty FHE (MPFHE) mitigates this by using MPC to establish communal keys, reducing the computation to that of single-key FHE while still supporting multiple users. However, in many cases [PS24,OT25,MTBH21] these amendments once again allow a single party and the server to collude and leak the private information of the honest participants in the protocol.

In this paper we present a framework which improves the security offered over the current state-of-the-art by preventing these sorts of semi-honest collusions between the parties and the server from impacting the security of the protocol. Additionally, the framework we present allows us to implement these constructions efficiently agnostic of the underlying MPC and FHE protocols.

First, the MPC protocol is used to generate all keys required for the FHE protocol to follow. Depending on the adopted FHE protocol the required key material can change, but in general we

³ There are methods of achieving active security through expensive verification mechanisms.

need two sets of key material: key switching keys and bootstrapping keys. As this only needs to be done once at the start of the protocol and can be re-used throughout the protocol independent of circuit size and multiple executions as long as the participants do not change the cost for this can be amortized. Secondly, the parties input using ciphertexts under their own individual keys, which the server can then key switch to a common key for the computation itself. Within the computation, the server can bootstrap using the bootstrapping keys, and parties can also supply additional inputs without further communication among the parties themselves, should that be needed. When outputting, the server can simply use the key switching keys for output, so the output ciphertext will be under the required key. Finally, the parties can now decrypt the output.

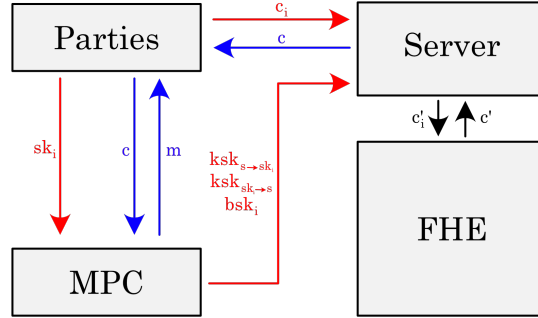


Fig. 1: A bird's eye overview of the MPFHE framework with encryption in red and decryption in blue .

For decryption we opt for a multi-key decryption protocol. After the completion of the final bootstrapping procedure, the server only does a single key switching to a multi-key LWE ciphertext. This also means the MPC preprocessing only requires one key switching key to be computed. However, the downside is that the parties must communicate for the final decryption as this needs to be done using a round-robin MPC protocol.

If one assumed a-priori that collusion between the server and parties was impossible, then an improvement can be made to our protocol: One could preprocess a shared encryption key, go through the whole process, and at the end the server could key switch from the shared encryption key to a single-key ciphertext using the keys of each party. This would require many key switching keys, many key switches, but decryption can now be done without communication. However, as this falls outside our security paradigm we will not treat this as a viable solution in our protocol.

Because this framework allows for the use of a single key FHE protocol over the multi-key variants, this directly combats the incredible noise growth and limitations that must be set on the number of participating parties.

1.1 Our Contributions

A Framework, Multiparty FHE: We introduce a new framework for multiparty fully homomorphic encryption (MPFHE) which allows for the use of any single key FHE protocol featuring linear decryption (e.g. BGV/BFV [BGV12,FV12], TFHE [CGGI20], FHEW [MS18,GPvL23], FINAL [BIP⁺22, JMPP24, PvLZ24]) in combination with a (semi-honest) MPC protocol to build a multiparty FHE scheme: namely one that requires some communication during key generation, but after

which the server can perform the computation with the same complexity as the single key FHE scheme.

An Instantiation, Multiparty FINAL: We instantiate our new framework by adapting FINAL [BIP⁺22] to the new framework and provide an improvement compared to FINALLY [PvLZ24] with a 2.46-71.16x reduction in total key material. We also provide an analysis of the noise growth and prove that this is still within the bounds of the lattice attacks known to affect NTRU.

An Implementation, Improving the State-of-the-Art: We provide a prototype implementation of MP-FINAL, which we use to benchmark our results against other the current state-of-the-art in Multi-user FINAL, [PvLZ24], Multi-party FHE, [AKÖ23,PR23], and Multi-Key FHE [KMS24].⁴

	PvLZ [PvLZ24]	PR [PR23]	KMS [KMS24]	AKÖ [AKÖ23]	This work
Key size (GB)	5337	2.2	.285	.93	.75
Bootstrapping time (ms)	-	900	5650	1800	88.6

Table 1: A brief overview of our results for $k = 16$ parties.

1.2 Related Work

To our knowledge, both our formalization of MPFHE and the structure of our framework have not appeared in literature before. Existing concepts in MPFHE, [MTBH21] and the static variant of [AKÖ23], do not consider the security when there is server-player collusion and variants of Threshold FHE, [PS24,OT25] explicitly disallow it. While [MTBH21] is the closest to the framework we propose, the protocol proposed is circuit dependent due to its reliance on BGV/BFV, [BGV12,FV12], rendering comparison between parameter sets impractical. Their bootstrapping also requires collective execution, whereas ours (after preprocessing) can be performed independently by the server. In the Threshold FHE case there is also a scheme that protects against collusions between server and players [CL25], however this scheme has a more widespread use of MPC, requiring it for every input. In our scheme, after the initial preprocessing to obtain keys, no further communication is required throughout the protocol even if additional inputs are required.

Generalizing our approach to a wider net does produce some interesting precursors: MPFHE can be interpreted as a relaxation of the multi-key FHE setting (MKFHE)[LTV12,PVZ24,CDKS19,Par21], where parties are now allowed communication during the setup phase. This has the result that the set of parties has to be, and stay, fixed before the start of the computation and key generation. It is clear that this communication will reduce the overall complexity of the protocols significantly and the proposed framework should translate to these MPFHE schemes without much work.

2 Preliminaries

2.1 Notation

Let R be a ring and let $\mathbb{F}$ be a field. For a polynomial, $g \in R[X]$ or $g \in \mathbb{F}[X]$, we denote by g_i the coefficient corresponding to the i th power of X . We denote vectors by lower-case bold letters, such

⁴ The code is available at <https://github.com/KULeuven-COSIC/vinyl>

that $\mathbf{a} = (a_1, \dots, a_n)$, and matrices by upper-case bold letters, $\mathbf{M} \in M_{r \times c}(\mathbb{F})$, such that $\mathbf{M}^{(i)}$ is the i th row of $\mathbf{M}$ and $\mathbf{M}_{(j)}$ is the j th column of $\mathbf{M}$. As both $\mathbf{M}^{(i)}$ and $\mathbf{M}_{(j)}$ are vectors a single index can be given by $M_j^{(i)} = M_{(j),i} = M_{i,j}$.

Throughout the paper we denote by $\mathcal{R} := \mathbb{Z}[X]/\langle X^N + 1 \rangle$ the $2N$ -th cyclotomic ring, such that $N = 2^\kappa$ for some $\kappa \in \mathbb{N}$. Similarly, we can define $\mathcal{R}_Q := \mathcal{R}/Q\mathcal{R}$, i.e. the cyclotomic ring with coefficients in $\mathbb{Z}_Q$. Note that any element $g \in R$, $R \in \{\mathcal{R}, \mathcal{R}_Q\}$, can be given as the smallest polynomial of at most degree N in the appropriate coset of R . Hence, we can represent any element, $g \in R$ by a vector representation, $\phi(g) = (g_0, \dots, g_{N-1})$, and this representation is well-defined. As $\text{Im}(\phi) \subset \mathbb{Z}^N$, we can define the infinity norm of an element, g , as $\|g\| = \|\phi(g)\|$.

2.2 Distributions

For a random variable a we define $a \leftarrow G_\sigma$ as the action of sampling a from the discrete Gaussian distribution over a ring R with variance σ^2 . Moreover, by $\text{Var}(a)$ we denote the maximum variance of a .

A random variable, V , is called α -subgaussian if the moment generating function fulfills the following equation for some α and all $t \in \mathbb{R}$:

$$\mathbb{E}[\exp(t \cdot V)] \leq \frac{1}{2} \exp(\alpha^2 \cdot t^2).$$

We will use the fact $\text{Var}(V) \leq \alpha^2$ and the following property for a vector $\mathbf{a} := (a_1, a_2, \dots, a_n)$ for our noise analysis:

$$\text{Var}(\mathbf{a}) = \max_{i \in [n]} (\text{Var}(a_i)) = \max_{i \in [n]} \alpha_i.$$

Furthermore, we employ the benefit of subgaussian variables called the Pythagorean Additivity property, which means that given an α -subgaussian distribution χ_α , a β -subgaussian distribution χ_β , $a \leftarrow \chi_\alpha$, $b \leftarrow \chi_\beta$, and any $t, s \in \mathbb{Z}$, it holds that $\text{Var}(t \cdot a + s \cdot b) = \sqrt{t^2 \cdot \alpha^2 + s^2 \cdot \beta^2}$. Lastly, for subgaussian random variables centered around zero, $a, b \in R[X]$, it holds that $\text{Var}(a \cdot b) \leq N \cdot \text{Var}(\phi(a)) \cdot \text{Var}(\phi(b))$.

A specific distribution relevant to our use of NTRU will be the FINAL distribution, which is $\frac{1}{2}$ -subgaussian:

$$\Pr(f_i = x) = \begin{cases} \frac{1}{4}, & x = -1 \\ \frac{1}{2}, & x = 0 \\ \frac{1}{4}, & x = 1 \end{cases}$$

2.3 Gadget Decomposition

For fixed integers Q, B , let $\ell = \lceil \log_B(Q) \rceil$. The ℓ -dimensional column vector defined by $\mathbf{g}_{Q,B} := (B^0, \dots, B^{\ell-1})$ is called the gadget vector. For any identity matrix $\mathbf{I}_k$ we can then define the gadget matrix $\mathbf{G}_{k,Q,B} = \mathbf{I}_k \otimes \mathbf{g}_{Q,B}$. To avoid confusion we will write $\mathbf{g} = \mathbf{g}_{Q,B} = \mathbf{G}_{1,Q,B}$ dropping parameters if they are clear from context.

The gadget decomposition associated with a particular gadget vector $\mathbf{g}$ is a function, denoted by $\mathbf{g}^{-1}$, such that for $x \in \mathbb{Z}_Q$, $\mathbf{g}^{-1}(x) = (x_0, \dots, x_{\ell-1})$ with $|x_i| < B/2$. For the notation as inverse to make sense, we require that $\mathbf{g}^{-1}(x) \cdot \mathbf{g} = x$ holds. The gadget decomposition naturally extends elementwise to vectors and polynomials.

2.4 Single-Key NGS

Within the bootstrapping, we will need single-key NGS, introduced in [BIP⁺22]. We introduce this scheme, as well as properties of the noise generation.

Definition 2.1 (Single-Key NGS). *Let λ be the security parameter. The NGS scheme consists of four probabilistic polynomial time algorithms: Setup, KeyGen, SEnc, VEnc.*

1. **NGS.Setup**(λ): Upon input of a security parameter λ , Setup returns the public parameters (N, Q, ζ, B, ℓ) .
2. **NGS.KeyGen**(ζ, N): Upon input of N and ζ , KeyGen samples $f' \leftarrow \chi_\zeta^N$ until $f^{-1} = (4 \cdot f' + 1)^{-1}$ exists in $\mathcal{R}_Q$. Then it outputs $\mathbf{sk} = f$.
3. **NGS.SEnc**_{sk}(m): Given a ternary polynomial m , sample $g \leftarrow \chi$, define $\Delta = \lfloor Q/4 \rfloor$, and output $c = g/f + \Delta m \in \mathcal{R}_Q$.
4. **NGS.VEnc**_{sk}(m): Given a ternary polynomial m , sample $g_i \leftarrow \chi^N$. Define $\mathbf{g} := (g_0, \dots, g_{\ell-1})$ and $\mathbf{g} = (B^0, \dots, B^{\ell-1})$. Output $c = \mathbf{g}/f + \mathbf{g}m \in \mathcal{R}_Q^\ell$.

The NGS scheme defines the external product of a scalar encryption $c = g/f + \Delta m$ and a vector encryption $\mathbf{c} = \mathbf{g}/f + \mathbf{g}m'$ as

$$c \boxtimes \mathbf{c} := \mathbf{g}^{-1}(c) \cdot \mathbf{c}.$$

Defining the noise of a scalar encryption c as $\text{err}(c) := c \cdot f - \Delta m$ and of a vector encryption as $\text{err}(\mathbf{c}) := \mathbf{c} \cdot f - \mathbf{g}m$, it can then be verified that the result of this operation is a scalar encryption of $m \cdot m'$ with noise term $\mathbf{g}^{-1}(c) \cdot \mathbf{g} + g \cdot m'$. If we introduce the notation γ^2 for the variance of the noise of a fresh encryption, we can furthermore quantify the noise growth when performing a sequence of external products.

Lemma 2.1 (Noise of a sequence of external products [BIP⁺22]). *Let $c_0 = g_0/f + \Delta \cdot m_0$ be an NGS scalar ciphertext encrypting a binary polynomial $m_0 \in \mathcal{R}_Q$, and let for each $i \in [k]$ $\mathbf{c}_i = \mathbf{g}_i/f + \Delta \cdot m_i$ be an NGS vector ciphertext encrypting $m_i \in \mathcal{R}_Q$. If $\text{ct} = c_0 \boxtimes_{i=1}^k \mathbf{c}_i$, then*

$$\text{Var}(\text{err}(\text{ct})) \leq N \cdot \ell \cdot \gamma^2 \cdot \sum_{i=1}^k \text{Var}(\mathbf{g}_i) + \text{Var}(g_0) + 4 \cdot \zeta^2.$$

Furthermore, if all ciphertexts are fresh, then

$$\text{Var}(\text{err}(\text{ct})) \leq (4 + ((k+1) \cdot N \cdot \ell \cdot \gamma^2)) \cdot \zeta^2.$$

2.5 Multi-key LWE

We recall here the multi-key FHE scheme based on the LWE-problem. We note that this simplifies to single-key LWE by selecting $k = 1$.

Definition 2.2 (Multi-Key LWE). *Let λ be the security parameter. Then, the Multi-Key LWE scheme consists of the following 5 PPT algorithms: Setup, KeyGen, Enc, Dec, and NAND.*

- **MKLWE.Setup**(λ): Upon input of a security parameter λ returns the public parameters (q, p, σ) . Denote by $\Delta := q/4$.
- **MKLWE.KeyGen**(σ): Upon input of the public parameters q, p , and σ , MKLWE.KeyGen randomly samples $s_i \in \mathbb{F}_2$, $i \in \{0, \dots, n-1\}$ such that $\mathbf{s} = (s_0, \dots, s_{n-1}) \in \mathbb{F}_2^n$.

- $\text{MKLWE.Enc}_s(m)$: Upon input of a key $\mathbf{s}$ and a message $m \in \mathbb{F}_2$, MKLWE.Enc generates $\mathbf{a} \leftarrow \mathbb{Z}_q^n$, and $e \leftarrow G_\sigma$. Then, it computes $b = \mathbf{a} \cdot \mathbf{s} + \Delta m + e$. It then outputs a vector of length $k + 1$, $\text{ct} = (b, 0, \dots, 0, \mathbf{a}, 0, \dots, 0)$ where $\mathbf{a}$ is in the position corresponding to the party generating the encryption, i.e. $\mathcal{P}_i$ is in the i th position.
- $\text{MKLWE.NAND}(\text{ct}_1, \text{ct}_2)$: Upon input of two ciphertexts of the form

$$\text{ct}_i = (b_i, \mathbf{a}_{i,1}, \dots, \mathbf{a}_{i,k}),$$

MKLWE.NAND outputs

$$\text{ct} = \left(\frac{5q}{8} - b_1 - b_2, \mathbf{a}_{1,1} + \mathbf{a}_{2,1}, \dots, \mathbf{a}_{1,k} + \mathbf{a}_{2,k} \right).$$

We denote a ciphertext encrypting m under multiple keys (such as $\text{sk}_1, \dots, \text{sk}_k$) as $\text{MKLWE.Enc}_{\text{sk}_1, \dots, \text{sk}_k}(m)$.

- $\text{MKLWE.Dec}(\mathbf{s}_1, \dots, \mathbf{s}_k, \text{ct})$: Upon input of keys $\mathbf{s}_1, \dots, \mathbf{s}_k$ and ciphertext ct of the form $(b, \mathbf{a}_1, \dots, \mathbf{a}_k)$ MKLWE.Dec computes

$$m^* = \left\lfloor \frac{b - \sum_{i=1}^k (\mathbf{a}_i \cdot \mathbf{s}_i)}{\Delta} \right\rfloor.$$

The MKLWE scheme is correct if

$$\text{MKLWE.Dec}(\text{sk}_1, \dots, \text{sk}_k, (\text{MKLWE.Enc}_{\text{sk}_1, \dots, \text{sk}_k}(m))) = m.$$

2.6 Key Switching

We briefly recall two forms of key switching: from (single-key) LWE and NGS to MKLWE, where we note that for both the instantiation of MKLWE with a single key constitutes a key switch to single-key LWE. We use the former in the input phase and the latter during the bootstrapping phase of section 4.

Key Switching from LWE to MKLWE Based on [CDKS21], we describe a key switching method from LWE to MKLWE. The key switching key generation is described in Figure 2, and the actual key switching then takes place in Figure 3. The noise generated by applying the key switching procedure can be quantified. Note that we generate a key switching key for each possible value of each $a_{i,j}$.

Lemma 2.2 ($\text{KS}_{\text{LWE} \rightarrow \text{MKLWE}}$ noise). *Let c be an LWE encryption of m and let c' be the output of $\text{KS}_{\text{LWE} \rightarrow \text{MKLWE}}$ with input c . Then, the following holds:*

$$\text{Var}(\text{err}(c')) \leq n \cdot \ell \cdot \text{Var}(\text{err}(\text{kss})) + \text{Var}(\text{err}(c)),$$

where $\text{Var}(\text{err}(\text{kss}))$ is that of a fresh ciphertext.

KSKGen _{LWE}
Input: LWE keys $\mathbf{s}, \mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_k$. Public parameters (n, q, B) . Output: Key switching keys $\text{ksk}_{i,j,v} := \text{MKLWE.Enc}_{\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_k}(v s_i B^j)$ for $i \in [n], j \in [\ell], v \in [B]$ with $\Delta = 1$

Fig. 2: LWE to MKLWE key switching key generation

KS _{LWE→MKLWE}
Input: LWE ciphertext under key $\mathbf{s}$: $\mathbf{c} = (b, \mathbf{a})$. Output of $\{\text{ksk}_{i,j,v}\}_{i,j,v} := \text{KSKGen}_{\text{LWE}}(\mathbf{s}, \mathbf{s}')$. Output: $(b', \mathbf{a}') \leftarrow (b, \mathbf{0}) - \sum_{i,j} \text{ksk}_{i,j,g^{-1}(\mathbf{a})_{i,j}}$

Fig. 3: LWE to MKLWE key switching

Key Switching from NGS to MKLWE For the second kind of key switching, we slightly modify the procedure from [BIP⁺22]. We define the key switching key generation procedure $\text{KSKGen}_{\text{NGS}}$ with the key switching procedure in Figure 4, and again quantify the noise growth of the operation. The same idea as before applies where we keep MKLWE encryptions of the NGS key with a gadget decomposition, and then rely on the linearity of decryption (for the 0th coefficient of the ciphertext). To generate the key switching key $\text{ksk}_{\text{NGS} \rightarrow \text{MKLWE}}$, from an NGS key f to an LWE key $\mathbf{s}$, sample the matrices $\mathbf{A}_i \leftarrow \mathbb{Z}_q^{(N \cdot L) \times n}$, $e \leftarrow \chi_\sigma$, define $\mathbf{f}_0 = (\phi(f)_0, -\phi(f)_{N-1}, -\phi(f)_{N-2}, \dots, -\phi(f)_1)$ and $\mathbf{P} = \mathbf{I}_N \otimes \mathbf{g}_{q, B_{\text{ksk}}}$. Then the output becomes $\text{ksk}_{\text{NGS} \rightarrow \text{MKLWE}} = (\mathbf{b} := \sum_{i \in [k]} \mathbf{A}_i \cdot \mathbf{s} + e + \mathbf{P} \cdot \mathbf{f}_0, \mathbf{A}_1, \dots, \mathbf{A}_k)$.

Lemma 2.3 ($\text{KS}_{\text{NGS} \rightarrow \text{MKLWE}}$ noise [BIP⁺22]). *Let $\mathbf{c}$ be a scalar NGS encryption of m with error e and $\mathbf{c}'$ be the output of $\text{KS}_{\text{NGS} \rightarrow \text{MKLWE}}$ with input $\mathbf{c}$. Then*

$$\text{Var}(\text{err}(\mathbf{c}')) \leq N \cdot L \cdot \gamma^2 \cdot \text{Var}(\text{err}(\text{ksk})) + \text{Var}(\text{err}(\mathbf{c})),$$

where N is the NGS dimension and $L = \log_B(Q)$ and $\text{Var}(\text{err}(\text{ksk}))$ is that of a fresh ciphertext.

KS _{NGS→MKLWE}
Input: $\mathbf{c}, \text{ksk}_{\text{NGS} \rightarrow \text{MKLWE}}$
1. Parse $(\mathbf{b}, \mathbf{A}) \leftarrow \text{ksk}_{\text{NGS} \rightarrow \text{MKLWE}}$ 2. $\mathbf{y} \leftarrow \mathbf{g}^{-1}(\phi(\mathbf{c}))$ 3. for i in $[k]$: (a) $\mathbf{a}_i \leftarrow \mathbf{y} \cdot \mathbf{A}_i$ 4. $b \leftarrow \mathbf{y} \cdot \mathbf{b}$ 5. Output $\mathbf{c}' := (b, \mathbf{a}_1, \dots, \mathbf{a}_k)$

Fig. 4: NGS to MKLWE key switching

2.7 Modulus Switching

We implement the method of modulus switching LWE ciphertexts as in [BIP⁺22]. Recall the definition of the randomized rounding function from [DM15].

Definition 2.3 (Randomized Rounding Function). Let $Q, q \in \mathbb{Z}$ with $1 < q < Q$. The randomized rounding function $[\cdot]_{Q,q} : \mathbb{Z}_Q \rightarrow \mathbb{Z}_q$ is defined as $[x]_{Q,q} = \lfloor q \cdot x/Q \rfloor + B$ where $B \in \{0, 1\}$ is a Bernoulli random variable with $\Pr[B = 1] = (q \cdot x/Q) - \lfloor q \cdot x/Q \rfloor$.

This extends element-wise to vectors, matrices and polynomials. Note that the rounding error is ϵ -subgaussian. Modswitching LWE ciphertexts is done by applying the Randomized Rounding Function elementwise to the ciphertext. Denote by $\text{ModSwitch}(c, Q, q) = [c]_{Q,q}$.

Lemma 2.4 (Modulus Switching Noise). For any LWE ciphertext $\mathbf{c}$ encrypting a message m under the key s with noise η and modulus q , then the output of $\text{ModSwitch}(\mathbf{c}, Q, q)$ is an LWE ciphertext $\mathbf{c}'$ encrypting the same message under the same key but with modulus Q and noise bounded by

$$\text{Var}(\text{err}(\mathbf{c}')) \leq \frac{q^2}{Q^2} \eta + (n + 1) \sqrt{2\pi}.$$

Proof. Let $\mathbf{c} = (b, \mathbf{a})$ and $\text{ModSwitch}(\mathbf{c}, Q, q) = (b', \mathbf{a}')$. For $i \in 1, \dots, n$, it holds that $a'_i = \frac{q}{Q} a_i + r_i$ and $b' = \frac{q}{Q} b + r_0$ for independent $\sqrt{2\pi}$ -subgaussian rounding errors $r_0, r_1, \dots, r_n$. The result follows in a straightforward manner.

2.8 Multiparty Computation

One of the key features of our framework is the use of a Secure Multiparty Computation (MPC) protocol to relieve, in large part or in full, the computationally complex Multi-Key FHE protocols in favor of a Single-Key FHE protocol. For this we will define an MPC functionality as an arithmetic black box. Leading up to the full functionality, we recall some important definitions of MPC.

The Adversary: Semi-honest and Full Threshold In MPC the adversary, $\mathcal{A}$, can be defined on several axes which define the powers of the adversary, [Lin20].

A first axis answers the question as to *how* the adversary behaves. In the case of *semi-honest* corruption, we assume that the adversary can not instruct corrupted parties to break away from protocol specification. This means that while the adversary gains access to all internal states of the corrupted parties it can only use those states to learn more about the underlying private information of honest parties. If the adversary exerts *malicious* corruption, however, it is allowed to force corrupted parties to behave against protocol description and deviate arbitrarily. For the purposes of this paper we will consider a semi-honest adversary. Due to the nature of FHE protocols and their malleability, a maliciously secure assumption on the MPC protocol would be too strong for what the FHE can later provide.

Another axis to consider is *how many* honest parties can be corrupted. Here we once again have two options: In the *honest majority* case at most $n/2$ parties can be corrupted, where n is the total number of parties. Alternatively, in the *full threshold* case we allow $n - 1$ parties to be corrupted at any time. Again, for the purposes of this paper we will consider the full threshold adversary, allowing corruption of all but one honest party.⁵

⁵ With some care and appropriate approaches to decryption, our protocols should generalize to arbitrary thresholds and even arbitrary access structures.

Secure Multiparty Computation With the behavior of the adversary in place, we can now turn to the MPC functionality that is secure against the semi-honest, full threshold adversary as defined.

The MPC protocol will be defined by five probabilistic polynomial time (PPT) algorithms: Init, Input, Add, Multiply, Output. The exact description of which is given in Figure 5, [DPSZ12].

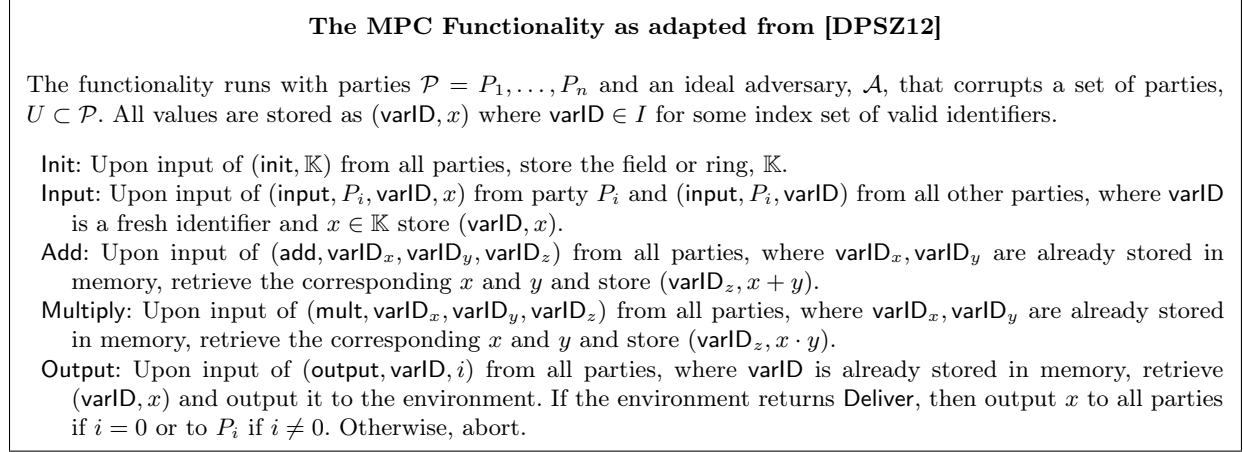


Fig. 5: The MPC Functionality, $\mathcal{F}_{\text{MPC}}$

Any MPC protocol that instantiates the MPC functionality, $\mathcal{F}_{\text{MPC}}$, correctly and obtains security against a semi-honest, full threshold adversary can be used as an instantiation of the following section. For optimization purposes, our key generation might be instantiated with “imperfect” and single-purpose MPC protocols. For ease of exposition, we do not cover these cases explicitly, but in many cases it will be possible to model these with only a resulting change in the noise of the MPC outputs, which can then be carried forward through the rest of our results.

3 A Framework for LWE-based Multiparty FHE Schemes

In this section, we demonstrate the creation of a multiparty FHE protocol from an MPC protocol and an sk-FHE protocol with very minimal restrictions on either. As in Section 2.8 we require that the MPC protocol is secure against a semi-honest, and full threshold adversary where the set of corruptible entities comprises the parties, $\mathcal{P}$, and the evaluator, $\mathcal{E}$. We will now introduce the functionalities of single key FHE, the multiparty functionality, and finally the multiparty protocol.

3.1 Single-Key FHE: Security versus Efficiency

One of the main benefits of the multiparty framework that we present is the ability to preprocess a large amount of the key generation using an MPC protocol. This allows us to generate a common key without any of the parties involved ever having access to this common key. However, this causes an issue to arise when we arrive at the decryption phase of the protocol; We can not key switch to the common key after the bootstrap as this is unknown to the parties involved. Hence, we must amend the standard single-key FHE functionality to include a key switching functionality to key switch from the unknown common key to the private keys of all parties. The functionality for this can be found in Figure 6.

The sk-FHE functionality with multi-key decryption, $\mathcal{F}_{\text{skFHE}+}$

The functionality runs with k parties $\mathcal{P}_1, \dots, \mathcal{P}_k$ and a semi-honest evaluator $\mathcal{E}$.

Init(): Upon receiving **Init** from the parties and the evaluator, the functionality initializes the database of single-key encryptions $\mathbb{S}$ and the database of key switching keys **KSK**.

KeyGen: Upon input of **KeyGen** without further inputs from $\mathcal{P}_i$, the functionality outputs an FHE key s to $\mathcal{P}$.

EvalKeyGen(s_i): Upon input of **EvalKeyGen**(s) by $\mathcal{P}_i$, the functionality generates k_{eval} and sends it to $\mathcal{E}$.

KSKGen(s, s'): Upon input of **KSKGen**(s, s'), the functionality stores the tuple (s, s', ksk) in **KSK**.

KS(c, ksk): Upon input of **KS**(c, ksk) where $(c, s, m) \in \mathbb{S}$ and $(s, s', \text{ksk}) \in \text{KSK}$, the functionality stores $(c', s', m) \in \mathbb{S}$ and outputs c' .

Enc_s(m): Upon input of **Enc_s**(m) from $\mathcal{P}$, send **Enc_s**(m) to $\mathcal{P}$.

Eval($f, k_{\text{eval}}, \text{inputs}$): Upon input of **Eval**($f, k_{\text{eval}}, \text{inputs}$) by $\mathcal{P}$, where **inputs** is a set of encryptions of inputs each under the same key s to the function f and k_{eval} was generated by **EvalKeyGen**(s), the functionality returns $f(m_1, \dots, m_n)$ to the parties.

Dec(c, s): Upon input of **Dec**(c, s), if there is a stored tuple (s, m, c) for some m , return m . Else, return $\perp$.

MKKSGen($s, \{s'_i\}_i$): Upon input of **MKKSGen**($s, \{s'_i\}_i$), the functionality stores the tuple $(s, \{s'_i\}_i, \text{ksk})$ in **KSK**.

MKKS(c, ksk): Upon input of **MKKS**(c, ksk) where $(c, s, m) \in \mathbb{S}$ and $(s, \{s'_i\}_i, \text{ksk}) \in \text{KSK}$, the functionality stores $(c', \{s'_i\}_i, m) \in \mathbb{S}$ and outputs c' .

MKDec($c, \{s_i\}_i$): Upon input of **MKDec**($c, \{s_i\}_i$) by a subset of parties $\mathcal{P} \subseteq \{\mathcal{P}_1, \dots, \mathcal{P}_k\}$ where each call has the same input ciphertext c but potentially different sets of keys $(\{s_i\}_i)_j$. If $(c, \{s_i\}_i, m) \in \mathbb{S}$ and $\{s_i\}_i = \cup_j (\{s_i\}_i)_j$, return m to all parties that called **MKDec** with c as an input.

Fig. 6: The single-key FHE Functionality with Multi-key Decryption

Any single-key FHE protocol extended with multi-key key switching and decryption which implements the functionality securely will provide security against semi-honest corruption of each of the parties and/or the server. As used later, the LWE protocol described in the preliminaries is a fit.

3.2 Generalized Multiparty FHE

In this section we will formulate the multiparty FHE functionality. After, we will show that this produces a secure protocol under specific, but not wildly restrictive, assumptions.

The multiparty FHE functionality contains 7 PPT algorithms: **Init**, **KeyGen_{MPC}**, **Dec**, **Input**, **Output**, **Eval**, and **Output**. Upon input of a set of parties, $\mathcal{P} = \mathcal{P}_1, \dots, \mathcal{P}_n$, an evaluator, $\mathcal{E}$, and an ideal adversary $\mathcal{A}$ that corrupts a set of parties, $U \subset (\mathcal{P} \cup \{\mathcal{E}\})$, $\mathcal{F}_{\text{mpFHE}+}$ can do the following:

- **Init** allows the functionality to initialize any variables that it may need. This includes, if necessary, the instantiation of the FHE and MPC protocols, as well as the key generation for the FHE protocol.
- **KeyGen_{MPC}** allows for the generation of the (unknown) communal key. As this is the main branch of the diverging paths we will discuss these in detail later.
- **Dec** behaves in line with FHE Decryption, taking in a ciphertext, $\mathbf{c}$, and a private key, $\mathbf{sk}$, or a set of private keys, $\{\mathbf{sk}_i\}_{i \in I}$, and outputs the underlying plaintext, m .
- **Input** formalizes the encryption and communication of an input to the evaluator, $\mathcal{E}$. The Evaluator also key switches to the unknown common key in this step.
- **Eval** allows the evaluator, $\mathcal{E}$, to evaluate a given function, f , on given inputs, $\{c_i\}_{i \in I}$.

The mp-FHE functionality with multi-key decryption, $\mathcal{F}_{\text{mpFHE}+}$

The functionality runs with parties $\mathcal{P} = P_1, \dots, P_n$, an evaluator $\mathcal{E}$ and an ideal adversary, $\mathcal{A}$, that corrupts a set of parties, $U \subset \mathcal{P}$. All values are stored as (varID, x) where $\text{varID} \in I$ for some index set of valid identifiers.

Init($\mathbb{K}$): Upon receiving **Init** from all parties and the evaluator, the functionality initializes the database of encryptions $\mathbb{S}$ and stores the field or ring, $\mathbb{K}$. Furthermore, when receiving **Init** from P_i , the functionality outputs an FHE key s_i to P_i .

KeyGen_{MPC}(s_i): Upon input of $(\text{KeyGen}_{\text{MPC}}, i, s_i)$ from each party, the functionality then generates and stores the required communal keys k_{com} . Furthermore, the functionality generates and outputs the input, output, and evaluation keys $k_{\text{input}}, k_{\text{eval}}, k_{\text{output}}$ to $\mathcal{E}$.

Input(c, s): Upon input of **Input**(c, s) from party P_i , generate some c , store $(s, m, c) \in \mathbb{S}$, return c to P_i , and send c to $\mathcal{E}$. The evaluator then generates c' and stores $(c', s', m) \in \mathbb{S}$, and outputs c' to $\mathcal{E}$.

Eval(f, k_{eval}, C): Upon input of **Eval**(f, k_{eval}, C) by $\mathcal{E}$ where f is some function, k_{eval} are the evaluation keys, and C is the set of ciphertexts to be used as inputs, such that $\forall c_i \in C, (s, c_i, m_i) \in \mathbb{S}$ with $s \in k_{\text{com}}$, and k_{eval} was generated by **EvalKeyGen**(s), the functionality stores $(c', s, f(m_1, \dots, m_n)) \in \mathbb{S}$ and sends c' to $\mathcal{E}$.

Output(c): Upon input of **Output**(c) by $\mathcal{E}$, the functionality checks whether $(s, c, m) \in \mathbb{S}$ with $s \in k_{\text{com}}$. If so, the functionality stores $(\{s_i\}_{i \in [k]}, c', m) \in \mathbb{S}$, outputs c' to $\mathcal{E}$. Then the functionality outputs c' to all players.

Dec(c, s_i): Upon input of **Dec**(c, s_i) by each P_i , if there is a stored tuple $(\{s_i\}_{i \in [k]}, c, m)$ for some m , return m to all parties.

***Fig. 7:** The Multiparty-FHE Functionality with Multi-key Decryption*

- **Output** formalizes the communication of an output ciphertext to the parties, $\mathcal{P}$. The evaluator performs key switching to multi-key under all of the parties' individual secret keys here.

The detailed functionality with multi-key decryption, $\mathcal{F}_{\text{mpFHE}+}$, can be found in Figure 7.

The communal key is generated by all parties and, as such, will use an MPC protocol in the implementation. The generation of this key is identical to the generation of a single-key FHE key, but keeping it secret the parties or the adversary. Alongside this, key switching keys from each of the private keys of the parties to this (unknown) communal key, and key switching keys from the communal key to a multi-key scheme under all parties' secret keys are generated. As the private keys of the parties never get shared and the communal key is not known to any of the parties, it and the key switching keys can be used repeatedly over executions. This means they only need to be generated once, allowing the scheme to amortize the preprocessing phase.

3.3 The Framework in Practice

In Figure 8, we provide a protocol implementing our mp-FHE functionality. We argue now that this construction faithfully implements the functionality and that the security of this construction only relies on the security of the underlying MPC and FHE protocols. We will make this argument by first arguing that $\Pi_{\text{mpFHE}+}$ implements $\mathcal{F}_{\text{mpFHE}+}$.

Consider $\Pi_{\text{mpFHE}+}$ and note that **Init**, **Input**, and **Output** are trivial combinations of calls to functions from $\mathcal{F}_{\text{MPC}}$ and $\mathcal{F}_{\text{skFHE}+}$. This means that the only two algorithms that remain are **Dec** and **KeyGen_{MPC}**.

The Dec Algorithm Upon input of (Dec, c, s_i) by a subset of parties, $A \in \mathcal{P}$, the parties instantiate $\mathcal{F}_{\text{MPC}}$ and run **MKDec** as described in $\mathcal{F}_{\text{skFHE}+}$ in MPC. As $\mathcal{F}_{\text{MPC}}$ receives a ciphertext, c_i , and a set of keys, $\{s_i\}_{P_i \in A}$ from each party it can first check if $c_i = c_j$ for all i, j . If this is not the case it aborts. Otherwise, it calls **MKDec** to see if the tuple $(c, \{s_i\}_{P_i \in A}, m)$ exists. As the parties

in A only hold their own keys, the only ciphertexts for which MKDec will contain such a tuple are those which (a subset of) the parties in A called to encrypt. Hence, they are also allowed to obtain the decryption. Otherwise, the parties in A will not hold the required keys to decrypt, and so $s_l \notin \{s_i\}_{P_i \in A}$ and so the tuple does not exist. In the latter case, the parties in A learn nothing as required. Hence, we can conclude that $\Pi_{\text{mpFHE}+}.\text{Dec}$ correctly implements $\mathcal{F}_{\text{mpFHE}+}.\text{Dec}$.

The Init Algorithm Upon calling Init in $\mathcal{F}_{\text{mpFHE}+}$, a party would expect a key s_i . When running the protocol, the party instead calls $\mathcal{F}_{\text{skFHE}+}.\text{KeyGen}$, which returns the same result.

The KeyGen_{MPC} Algorithm Upon input of $(\text{KeyGen}_{\text{MPC}}, i, s_i)$ by each party $\mathcal{P}_i$ in the functionality, the expected output is that the functionality stores the communal key s , and outputs k_{input} , k_{eval} , and k_{output} to the server $\mathcal{E}$.

In the protocol however, the parties all run the MPC protocol. We argue that collectively using the MPC protocol to generate the secret key and not outputting it is equivalent to the functionality storing the secret key. There are three places where the functionality uses the stored key s : $\text{KS}_{\text{toComm}}$, $\text{KS}_{\text{fromComm}}$, and Eval . Each of these is equivalent to the server possessing the correct keys used during the appropriate calls to $\mathcal{F}_{\text{skFHE}+}$. In $\text{KS}_{\text{toComm}}$ the check is equivalent to the server possessing the right key switching key in the protocol as the only way to perform $\text{KS}_{\text{toComm}}$ is to pass $\mathcal{F}_{\text{skFHE}+}.\text{KS}$ a key switching key to the communal key s . Such keys are also generated and stored by $\mathcal{E}$ in k_{input} . In $\text{KS}_{\text{fromComm}}$, the check is likewise equivalent to $\mathcal{E}$ possessing the correct ksk for the call to $\mathcal{F}_{\text{skFHE}+}.\text{MKKS}$ with a ksk from k . The only such key is stored in k_{output} , generated during the MPC protocol. Finally, in Eval , the check is equivalent to the server possessing k_{eval} as this is generated using $\mathcal{F}_{\text{skFHE}+}.\text{EvalKeyGen}$ called in MPC, with s as input.

The generation of the remaining keys is indistinguishable, as will be demonstrated in the algorithm in which they are used.

The Input Algorithm First, in the functionality, the party receives a ciphertext corresponding to the input message and key. In the protocol, the party calls $\mathcal{F}_{\text{skFHE}+}.\text{Enc}$ to receive the same value.

In both cases, the server receives this ciphertext.

Then in the functionality, this protocol returns a key switched ciphertext to the server if and only if (s, c, m) is a stored ciphertext and that $s \notin k_{\text{com}}$. If so, (s', c', m) is stored and c' is output to $\mathcal{E}$.

In the protocol, the check that (s, c, m) is a stored ciphertext is performed by $\mathcal{F}_{\text{skFHE}+}.\text{KS}$ and the check that $s \notin k_{\text{com}}$ is equivalent to $\mathcal{E}$ passing the correct ksk as $\mathcal{E}$ will possess this key (and thus the check $(s, s', \text{ksk}) \in \text{KSK}$ will pass) if and only if this ksk is generated in $\text{KeyGen}_{\text{MPC}}$ as generating others outside the protocol would require the knowledge of s , which $\mathcal{E}$ does not have. The only such keys generated are those to s , the communal key.

The Output Algorithm In the functionality, this algorithm returns a key switched ciphertext to the server if and only if (s, c, m) is a stored ciphertext and that $s \in k_{\text{com}}$. If so, $(\{s_i\}_i, c', m)$ is stored and c' is output to $\mathcal{E}$.

In the protocol, the check that (s, c, m) is a stored ciphertext is performed by $\mathcal{F}_{\text{skFHE}+}.\text{MKKS}$ and the check that $s \in k_{\text{com}}$ is equivalent to $\mathcal{E}$ passing the correct ksk as $\mathcal{E}$ will possess this key (and thus the check $(s, \{s_i\}_i, \text{ksk}) \in \text{KSK}$ will pass) if and only if this ksk is generated in $\text{KeyGen}_{\text{MPC}}$.

The mp-FHE protocol with multi-key decryption, $\Pi_{\text{mpFHE}+}$ in the $\mathcal{F}_{\text{skFHE}+}, \mathcal{F}_{\text{MPC}}$ -Hybrid Model

The protocol runs with parties $\mathcal{P} = P_1, \dots, P_k$ and an evaluator $\mathcal{E}$. All values are stored as (varID, x) where $\text{varID} \in I$ for some index set of valid identifiers.

Init($\mathbb{K}$): The parties call $\mathcal{F}_{\text{MPC}}.\text{Init}(\mathbb{K})$ and $\mathcal{F}_{\text{skFHE}+}.\text{Init}$. Each party calls $\mathcal{F}_{\text{skFHE}+}.\text{KeyGen}$.

KeyGen(s_i): The parties run $\mathcal{F}_{\text{MPC}}.\text{Input}$, followed by a mix of $\mathcal{F}_{\text{MPC}}.\text{Add}$ and $\mathcal{F}_{\text{MPC}}.\text{Multiply}$ such that the following is computed:

- $s \leftarrow \mathcal{F}_{\text{skFHE}+}.\text{KeyGen}$
- for $i \in [k]$:
 - $\text{ksk}_i \leftarrow \mathcal{F}_{\text{skFHE}+}.\text{KSKGen}(s_i, s)$
- $k_{\text{output}} \leftarrow \mathcal{F}_{\text{skFHE}+}.\text{MKKSKGen}(s, \{\text{ksk}_i\}_{i \in [k]})$
- $k_{\text{eval}} \leftarrow \mathcal{F}_{\text{skFHE}+}.\text{EvalKeyGen}$.

Finally, the parties run $\mathcal{F}_{\text{MPC}}.\text{Output}$ so that $\mathcal{E}$ gets $k_{\text{input}} := \{\text{ksk}_i\}_i, k_{\text{eval}}, k_{\text{output}} := \{\text{ksk}'_i\}_i$.

Input(c): The party calls $\mathcal{F}_{\text{skFHE}+}.\text{Enc}_s(m)$ which outputs c . The party sends c to $\mathcal{E}$. $\mathcal{F}_{\text{skFHE}+}.\text{KS}(c, \text{ksk})$ where ksk is the appropriate element of k_{input} .

Eval(f, k_{eval}, C): The evaluator calls $\mathcal{F}_{\text{skFHE}+}.\text{Eval}(f, k_{\text{eval}}, C)$.

Output(c): The evaluator calls $\mathcal{F}_{\text{skFHE}+}.\text{MKKS}(c, \text{ksk})$ where ksk is the appropriate element of k_{output} . This outputs c' to the evaluator, who then sends c' to $\mathcal{P}_i$.

Dec(c, s): The parties instantiate $\mathcal{F}_{\text{MPC}}$ to execute $\mathcal{F}_{\text{skFHE}+}.\text{MKDec}(c, s_{ii})$ and output to all parties.

Fig. 8: The Multiparty-FHE Protocol With Multi-key Decryption

as generating others outside the protocol would require the knowledge of s , which $\mathcal{E}$ does not have. The only such keys generated are those from s , the communal key.

Finally, in both cases, all parties receive the output ciphertext.

The Eval Algorithm In the functionality, this algorithm returns c' encrypting $f(m_1, \dots, m_n)$ with the appropriate data stored if and only if all input ciphertexts c_i are in $\mathbb{S}$ with the same associated key s , and k_{eval} was generated by $\mathcal{F}_{\text{mpFHE}+}.\text{EvalKeyGen}$ using s as input.

In the protocol, the same checks are performed by $\mathcal{F}_{\text{skFHE}+}.\text{Eval}$, and the same things are stored and returned by the same call to $\mathcal{F}_{\text{skFHE}+}.\text{Eval}$.

To show that security holds note that $\Pi_{\text{mpFHE}+}$ inherits most of its security guarantees from the hybrid construction and the underlying $\mathcal{F}_{\text{MPC}}$ and $\mathcal{F}_{\text{skFHE}+}$. The only consideration that is left to make are the generation and handling of the common key. The main issue is that we generate the secret key that is used to perform the entire computation in MPC. However, as this value is intermediate within the MPC, the only way to learn it is to either have all parties collude, which is impossible as at most $n - 1$ parties can be corrupt, or to extract the key from the key switching keys. Since the key switching keys are opened only to the evaluator, $\mathcal{E}$, who doesn't simultaneously know all of the parties' private keys this would break the security of $\mathcal{F}_{\text{skFHE}+}$.

3.4 Practical Considerations for the Choice of MPC and FHE Schemes

There are a number of considerations to consider when choosing the MPC and FHE protocols which realize the functionalities given above in a practical setting.

The FHE protocol needs to have an efficient key switching protocol of which the keys do not depend on the ciphertext, as the generation of keys midway through the FHE protocol would require another execution of the MPC protocol. In order to have security against a server colluding with a player, i.e. $\mathcal{F}_{\text{mpFHE}+}$, the FHE scheme needs a sister multi-key protocol or the ability to support multi-key key switching and decryption with specialized dedicated algorithms. A

general class of FHE protocols that support this includes the linear decryption FHE protocols, [COS⁺22], such as BGV [BGV12], BFV [FV12], and TFHE [CGGI16,CGGI20], as well as FINAL [BIP⁺22,JMPP24,PvLZ24] in particular.

The MPC protocol has far less stringent requirements on it. All we require is a full threshold protocol which is secure against semi-honest adversaries. Security against a malicious adversary would only provide marginal benefits, as a single malicious entity, server or party, could perform a trivial denial of service attack against the FHE protocol. Examples of these (and overperforming) are plentiful, e.g. GMW [GMW91], SPDZ [DPSZ12], and BGW [BGW88].

4 A Case Study: Multiparty FINAL

To exemplify the above generic construction $\Pi_{\text{mpFHE}+}$, we now present an adaptation of FINAL [BIP⁺22] to the multiparty setting as a specific instantiation. In this setting, we also show some of the optimization techniques that may become possible when the underlying single-key FHE scheme is no longer a black box. In section 5, we then implement this instantiation and evaluate how it scales with the number of parties.

4.1 Key Generation

We use MPC during preprocessing to compute the keys used during the protocol. Note that while some communal LWE and NGS keys theoretically exist, we never reveal these values. Instead, we compute the key switching keys from the individual party's LWE keys to the communal LWE key, as the key switching keys from the communal NGS ciphertext to the communal LWE key, and the key switching key from the single communal LWE key to multi-key LWE under all of the participating parties' keys.

In our protocol, we require the generation of several keys during the MPC:

- For each party $\mathcal{P}_i$ where $i \in [k]$, the key switching key from their LWE key $\mathbf{s}_i$ to the communal LWE key $\mathbf{s}$.
- The key switching key from the communal NGS key f to the communal LWE key $\mathbf{s}$.
- The bootstrapping keys bsk_i for $i \in [n]$ which allows for an indirect key switch from sk-LWE to NGS.
- The key switching key from the communal LWE key $\mathbf{s}$ to multi-key LWE under all parties' keys $\mathbf{s}_1, \dots, \mathbf{s}_k$.

To compute the bootstrapping keys, we run the algorithm in Figure 9 in MPC to generate the keys listed above. We note that the communal secret keys $\mathbf{s}$ and f are known to no-one, as these are never output from the MPC. Thus, the server, using the keys generated by this process, has the ability to do the following without compromising security:

- key switch a ciphertext of the form $\text{LWE.Enc}_{\mathbf{s}_i}(m)$ to $\text{LWE.Enc}_{\mathbf{s}}(m)$, used during the input phase.
- key switch a ciphertext of the form $\text{NTRU.Enc}_f(m)$ to $\text{LWE.Enc}_{\mathbf{s}}(m)$, used during the bootstrapping.
- key switch a ciphertext of the form $\text{LWE.Enc}_{\mathbf{s}}(m)$ to $\text{MKLWE.Enc}_{\mathbf{s}_1, \dots, \mathbf{s}_k}(m)$.

The noise of the key switching keys is that of a fresh ciphertext, see section 2.5.

Once the parties have generated their shares of the key switching keys ksk_i , they then send this key switching key to the server along with their input encrypted under their own LWE key $\mathbf{s}_i$. The server then key switches to the communal key $\mathbf{s}$ using the key switching from single-key LWE to single-key LWE in section 2.6 with key switching key ksk_i .

The Bootstrapping Key Generation Algorithm, Π_{KeyGen}

Input: Each party $\mathcal{P}_i$ inputs their LWE key $\mathbf{s}_i$. The Security parameter λ .

Output: The key switching keys ksk_i from $\mathbf{s}_i$ to $\mathbf{s}$ for all $i \in [k]$.

The key switching key $\text{ksk}_{\text{NTRU} \rightarrow \text{LWE}}$ from f to $\mathbf{s}$.

The key switching key ksk_{out} from $\mathbf{s}$ to $\mathbf{s}_1, \dots, \mathbf{s}_k$.

Algorithm:

1. $(N, Q, \zeta, B, l) \leftarrow \text{NGS.Setup}(\lambda)$
2. $(q, p, \sigma) \leftarrow \text{MKLWE.Setup}(\lambda)$
3. $\mathbf{s} \leftarrow \text{LWE.KeyGen}(\sigma)$.
4. $f \leftarrow \text{NGS.KeyGen}(\zeta, N)$.
5. $\text{ksk}_{\text{NTRU} \rightarrow \text{LWE}} \leftarrow \text{KSKGen}_{\text{NGS}}(f, \mathbf{s})$
6. for $i \in [k]$ do:
 - (a) $\text{ksk}_i \leftarrow \text{KSKGen}_{\text{LWE}}(\mathbf{s}_i, \mathbf{s})$.
7. for i in $[n]$ do:
 - (a) $\mathbf{g}_i \xleftarrow{\$} R_n$
 - (b) $\text{bsk}_i \leftarrow \frac{\mathbf{g}_i}{f} + \mathbf{g}\mathbf{s}_i$.
8. $\text{ksk}_{\text{out}} \leftarrow \text{KSKGen}_{\text{MKLWE}}(\mathbf{s}, (\mathbf{s}_1, \dots, \mathbf{s}_n))$.
9. Output $\text{ksk}_i, \text{ksk}_{\text{out}}, \text{ksk}_{\text{NTRU} \rightarrow \text{LWE}}$.

Fig. 9: The algorithm for computing the bootstrapping keys

4.2 Bootstrapping

In this section we introduce the bootstrapping using LWE as a base scheme as alluded to in [BIP⁺22]. The bootstrapping has the following structure. First, there's a minor step doing some preprocessing. Second, a blind rotation is done using CMux gates. This homomorphically decrypts the preprocessed ciphertext, providing an NGS ciphertext of the same message. Third, this NGS ciphertext is key switched back to an LWE ciphertext. Finally, the modulus is switched back to the LWE modulus from the NGS modulus. Each step is covered in more detail in what follows.

The Bootstrapping Algorithm, Π_{BS}

Input: $\text{ct} = (b, \mathbf{a})$, an LWE ciphertext encrypting m . The output of $\text{KSKGen}_{\text{NGS} \rightarrow \text{LWE}}(f, \mathbf{s}) =: \text{ksk}_{\text{NGS} \rightarrow \text{LWE}}$.

Output: ct' , an LWE ciphertext encrypting the same m .

1. $b \leftarrow \left\lfloor \frac{2 \cdot N \cdot b}{q} \right\rfloor$
2. $\mathbf{a} \leftarrow \left(\left\lfloor \frac{2 \cdot N \cdot (-\mathbf{a})}{q} \right\rfloor \right)$
3. $\text{ACC} \leftarrow \left\lfloor \frac{Q}{8} \right\rfloor \cdot X^{N/2} \cdot \sum_{i=0}^{N-1} X^i$
4. $\text{ACC} \leftarrow \text{ACC} \boxplus \mathbf{g} \cdot X^b$
5. for $i \leftarrow 0$ to $n-1$ do
 - (a) $c_{\text{mux}} \leftarrow \text{CMux}_i(\mathbf{a}_i)$
 - (b) $\text{ACC} \leftarrow \text{ACC} \boxplus c_{\text{mux}}$
6. $\text{ACC} \leftarrow \text{ACC} + \left\lfloor \frac{Q}{8} \right\rfloor \cdot \sum_{i=0}^{N-1} X^i$
7. $\text{ACC} \leftarrow \text{ModSwitch}(\text{ACC}, Q, q)$
8. $\text{ct}' \leftarrow \text{K}_{\text{NGS} \rightarrow \text{LWE}}(\text{ACC}, \text{ksk}_{\text{NGS} \rightarrow \text{LWE}})$

Fig. 10: Bootstrapping a Single-Key LWE Ciphertext

As in [BIP⁺22] and [PvLZ24], the first step of the bootstrapping is to convert the input LWE ciphertext to an NGS ciphertext. This is done via a series of binary CMux gates, described in

[CGGI20]. The binary CMux is defined as

$$\text{CMux}_i(c) = \mathbf{g} + (X^c - 1) \cdot \text{NGS.VEnc}_f(s_i).$$

Note that $\mathbf{g}$ is a noiseless NGS encryption of 1, and recall that $\text{bsk}_i = \text{NGS.VEnc}_f(s_i)$. We note that

$$\text{CMux}_i(c_i) = \mathbf{g}(1 + X^{c_i} s_i - s_i) + \frac{\mathbf{g}}{f_i}(X^{c_i} - 1) = \text{NGS.VEnc}_f(X^{c_i \cdot s_i}),$$

with noise $\mathbf{g}(X^{c_i} - 1)$. The last equality holds due to the following:

- If $s_i = 0$, then $X^{c_i s_i} = X^0 = 1$ and $1 + X^{c_i} s_i - s_i = 1$.
- If $s_i = 1$, then $X^{c_i s_i} = X^{c_i}$ and $1 + X^{c_i} s_i - s_i = X^{c_i}$.

As $c_i \in \mathbb{F}_2$, $X^{c_i s_i} \in \mathbb{M}$. We give the following lemma to bound the noise generated by a CMux gate.

Lemma 4.1. *Let $E_{\text{bsk}_i} = \text{Var}(\text{err}(\text{bsk}_i))$ and let E_{CMux_i} be the noise of the output of the CMux gate. Then $E_{\text{CMux}_i} \leq 2 \cdot E_{\text{bsk}_i}$.*

Proof. Follows from lemma 7 of [PvLZ24].

Given as input the LWE ciphertext $\text{LWE.Enc}_s(m)$ parsed as $(b, \mathbf{a})$, the first step of the bootstrapping is to preprocess this to obtain:

$$\begin{aligned} c_0 &:= \left\lfloor \frac{2 \cdot N \cdot b}{q} \right\rfloor, \\ \mathbf{c}_1 &:= \left\lfloor \frac{2 \cdot N \cdot (-\mathbf{a})}{q} \right\rfloor. \end{aligned}$$

We now would like to perform the following multiplication:

$$\mathbf{g} \cdot X^{c_0} \cdot \prod_{i \in [k]} \text{CMux}_i(c_{1,i}) = \text{NGS.VEnc}_f(X^{\Delta \cdot m + e}),$$

where we note that $\mathbf{g} \cdot X^{c_0} = \text{NGS.VEnc}(X^{c_0})$ with noise 0. The difficulty in performing such a multiplication is that there's no way to directly perform multiplications between NGS vector encryptions. Thus, we add a scalar NGS encryption called the accumulator on the left side of the computation to allow for the use of the external product defined in section 2.4.

The accumulator is defined as

$$\text{ACC} := \left\lfloor \frac{Q}{8} \right\rfloor \cdot X^{N/2} \cdot \sum_{i=0}^N X^i \mod (X^N + 1).$$

Using this well-known technique [GPvL23, MS18, CGGI20], the following lemma can be proven:

Theorem 4.1 ([BIP⁺22] Section 5.1). *Let $\text{ct} = (b, \mathbf{a})$ and let $c_0, \mathbf{c}_1, \text{bsk}_i$, and ACC be defined as above. Then*

$$c' := (\text{ACC} \boxtimes c_0) \boxtimes_i \text{CMux}_i(c_{1,i}) + \left\lfloor \frac{Q}{8} \right\rfloor \cdot \sum_{i=1}^{N-1} X^i = \text{NGS.SEnc}_f(m),$$

with $\Delta = \frac{Q}{4}$, such that $\text{Var}(\text{err}(c')) \leq (n+1) \cdot N \cdot l \cdot \gamma^2$.

Proof. First, we prove correctness, and after we will bound the noise generated. Recall that the accumulator is given by the following:

$$\text{ACC} = \left\lfloor \frac{Q}{8} \right\rfloor \cdot X^{N/2} \cdot \sum_{i=0}^{N-1} X^i.$$

Then using the external product in the first step obtains

$$\text{ACC} \boxtimes \mathbf{g} \cdot X^b = \left\lfloor \frac{Q}{8} \right\rfloor \cdot X^{N/2+b} \cdot \sum_{i=0}^{N-1} X^i.$$

Then applying the CMux once more for each $i \in [n]$ results in

$$\text{ACC} = \left\lfloor \frac{Q}{8} \right\rfloor \cdot \sum_{i=0}^{N-1} X^i \cdot X^{\lfloor \frac{2N}{q} b \rfloor} \cdot \prod_{i=1}^n X^{\lfloor \frac{2N}{q} a_i s_i \rfloor}.$$

Note that $b - \sum_{i=0}^n a_i s_i = \Delta \cdot m + e$. Since after applying a boolean gate, $\Delta = \lfloor \frac{q}{2} \rfloor$, the following holds:

$$\frac{2 \cdot N}{q} (\Delta \cdot m + e) = N \cdot (m + \frac{e}{\Delta}).$$

Due to our assumptions regarding the error bound, i.e. that $\lfloor \frac{e}{\Delta} \rfloor = 0$, we obtain:

$$\text{ACC} = \left\lfloor \frac{Q}{8} \right\rfloor \cdot X^{N/2} \cdot \sum_{i=0}^{N-1} X^i \cdot (-1)^m.$$

We are only interested in the value of the 0th coefficient. Observe that the 0th coefficient is at $i = N/2$, obtaining $X^{N/2} \cdot X^{N/2} = X^N = -1$. This results in

$$\text{ACC} = \left\lfloor \frac{Q}{8} \right\rfloor - 1 \cdot (-1)^m.$$

Note that this is a valid encryption $\text{NGS.SEnc}(2m-1)$. By adding a noiseless encryption of 1 using $\Delta = \lfloor Q/8 \rfloor$, and observing that in the remainder we use $\Delta = \lfloor Q/4 \rfloor$, we obtain:

$$\text{ACC} = \text{NGS.SEnc}(m)$$

as desired.

We now bound the noise. As there are $n+1$ total CMux multiplications, we obtain from Lemma 2.1 that the noise of this sequence is bounded by

$$\text{Var}(\text{err}(c')) \leq (n+1) \cdot N \cdot l \cdot \gamma^2 \cdot \text{Var}(\text{err}(\text{CMux}_i(a_i))) + \text{Var}(\text{err}(T(X))).$$

By Lemma 4.1, note that $\text{Var}(\text{err}(\text{CMux}_i(a_i))) \leq 2 \cdot \text{Var}(g)$, thus:

$$\text{Var}(\text{err}(c')) \leq (n+1) \cdot N \cdot l \cdot \gamma^2.$$

Now that we have the results from Theorem 4.1, we can key switch from NGS to LWE using the key switching protocol from [BIP⁺22], recalled in section 2.6. Doing so produces an LWE ciphertext of the same message as was input, however this encryption is under the NGS modulus Q .

Thus, we must still switch back to the LWE modulus q before we once again have a ciphertext of the same form as the input of the bootstrapping. We use the modulus switching from section 2.7.

Now the bootstrapping is complete, the following holds:

Theorem 4.2. *Let $\mathcal{P}$ be the set of k participating parties. Let $\mathbf{c} := \text{LWE.Enc}_s(m)$. Then Π_{BS} outputs $\mathbf{c}' = \text{LWE.Enc}_s(m)$ with noise bounded by:*

$$\begin{aligned} \text{Var}(\text{err}(\Pi_{\text{BS}}(\mathbf{c}))) &= \text{Var}(\text{err}(\Pi_{\text{KS}}(\Pi_{\text{MS}}(\text{ACC})))) \\ &\leq N \cdot L \cdot \gamma^2 \cdot \sigma^2 + \frac{q^2}{Q^2} \cdot (n+1) \cdot N \cdot l \cdot \gamma^2 + (n+1) \cdot \sqrt{2\pi}. \end{aligned}$$

Proof. By theorem 4.1, the error after step 6 is given by

$$\text{Var}(\text{err}(\text{ACC})) \leq (n+1) \cdot N \cdot l \cdot \gamma^2.$$

As this is the final step using the NGS scheme, this is also the bound for the noise while working in this scheme.

By lemma 2.4, it is known that the noise introduced by the modulus switching algorithm is bounded by:

$$\begin{aligned} \text{Var}(\text{err}(\Pi_{\text{MS}}(\text{ACC}))) &\leq \frac{q^2}{Q^2} (\text{Var}(\text{err}(\text{ACC})) + (n+1)\sqrt{2\pi}) \\ &\leq \frac{q^2}{Q^2} \cdot (n+1) \cdot N \cdot l \cdot \gamma^2 + (n+1)\sqrt{2\pi}. \end{aligned}$$

Thus, via lemma 2.3 the theorem is proven:

$$\begin{aligned} \text{Var}(\text{err}(\Pi_{\text{BS}}(\mathbf{c}))) &= \text{Var}(\text{err}(\Pi_{\text{KS}}(\Pi_{\text{MS}}(\text{ACC})))) \\ &\leq N \cdot L \cdot \gamma^2 \cdot \text{Var}(\text{err}(\text{ksk})) + \text{Var}(\text{err}(\Pi_{\text{MS}}(\text{ACC}))) \\ &\leq N \cdot L \cdot \gamma^2 \cdot \sigma^2 + \frac{q^2}{Q^2} \cdot (n+1) \cdot N \cdot l \cdot \gamma^2 + (n+1) \cdot \sqrt{2\pi}. \end{aligned}$$

4.3 Heuristics

Denote by $E_{\text{BS}} = \text{Var}(\text{err}(\Pi_{\text{BS}}(\mathbf{c})))$. This noise, obtained after bootstrapping, behaves as a Gaussian distribution. Thus, we can apply the Central Limit Theorem and its heuristic to conclude that $\|\text{err}(\Pi_{\text{BS}}(\mathbf{c}))\| \leq 6 \cdot \sqrt{E_{\text{BS}}}$.

From Theorem 4.2, we can conclude that

$$\|\text{err}(\text{ct}_{\text{NGS}})\| \leq 6 \cdot \sqrt{N \cdot L \cdot \gamma^2 \cdot \sigma^2 + \frac{q^2}{Q^2} \cdot (n+1) \cdot N \cdot l \cdot \gamma^2 + (n+1) \cdot \sqrt{2\pi}}. \quad (1)$$

Theorem 4.3. *If the NGS ciphertexts of the protocol in Figure 10 satisfy equation 1 except with negligible probability and $Q = \mathcal{O}(N \log(N))$, then these ciphertexts can be decrypted, except with negligible probability.*

Proof. As $n = \mathcal{O}(N)$, $\{l, L\} = \mathcal{O}(\log Q) = \mathcal{O}(\log N)$ and $\{\gamma^2, \sigma^2\} \in \mathcal{O}(1)$, we have that the noise of NGS ciphertexts in the protocol in Figure 10 are $\mathcal{O}(N \log(N))$ except with negligible probability by Eq 1. We require this noise to be less than $Q/4$ for correct decryption, thus it is sufficient to select $Q = \mathcal{O}(N \log(N))$.

4.4 Decryption

After bootstrapping, the server holds an LWE ciphertext of the output encrypted under s . If this isn't the end of the protocol, the server then continues the computation, bootstrapping as needed. If this is the end of the protocol, then the server must perform keyswitching from the common key to a multi-key ciphertext under the individual keys for all the parties, before sending the output ciphertext back to the parties.

Theorem 4.4. *Let $\mathbf{c}$ be the output of a freshly bootstrapped ciphertext and let $\mathbf{c}'$ be the output of $\text{KS}_{\text{LWE} \rightarrow \text{MKLWE}}(\mathbf{c}, \text{ksk})$. Then:*

$$\text{Var}(\text{err}(\mathbf{c}')) \leq 2 \cdot N \cdot L \cdot \gamma^2 \cdot \sigma^2 + \frac{q^2}{Q^2} \cdot (n+1) \cdot N \cdot l \cdot \gamma^2 + (n+1) \cdot \sqrt{2\pi}.$$

Furthermore, $\mathbf{c}'$ decrypts correctly with overwhelming probability if $q = \mathcal{O}(n^2)$

Proof. The noise bound follows directly from Theorem 4.2 and Lemma 2.2. We use the Central Limit Theorem heuristic to conclude that

$$\|\text{err}(\mathbf{c}')\| \leq 6 \cdot \sqrt{\text{Var}(\text{err}(\mathbf{c}'))}.$$

As $N = \mathcal{O}(n)$, $\{l, L\} = \mathcal{O}(\log n)$, $\left\{\frac{q^2}{Q^2}, \gamma, \sigma\right\} = \mathcal{O}(1)$,

$$\|\text{err}(\mathbf{c}')\| = \mathcal{O}(n \log n).$$

The correct decryption follows as $\mathcal{O}(n \log n) < \mathcal{O}(n^2)$.

A Note on Breaking the Black Box We also note that an improvement can be made to this instantiation by breaking the black box of the decryption. In this improvement, instead of generating a keyswitching key from sk-LWE to mk-LWE, you instead have the parties generate a keyswitching key from sk-NGS to mk-LWE, and then replace the keyswitch in the final bootstrapping with a keyswitch to mk-LWE instead. This saves the effort of an extra keyswitch. However, we choose to not do this to faithfully implement our functionality.

4.5 Key Compression

We observe that in practice, storing the key switching keys comes at a noticeable cost in memory usage. Storing a single $\text{ksk}_{\text{LWE} \rightarrow \text{LWE}}$ incurs a total of $n \cdot \ell \cdot B \cdot (n+1) \cdot \log q$ bits of storage, with $\text{ksk}_{\text{NTRU} \rightarrow \text{LWE}}$, $\text{ksk}_{\text{NTRU} \rightarrow \text{MKLWE}}$ and $\text{ksk}_{\text{LWE} \rightarrow \text{MKLWE}}$ following similar patterns. Following ideas from [CDKS21, CNT12], we can compress the constituent LWE ciphertexts by sampling a single random seed of length λ that can be expanded into the (public, uniformly random) $\mathbf{a}$ component of the ciphertext. This in turn leads to only $n \cdot \ell \cdot B \cdot \log q + \lambda$ bits of storage needed for a $\text{ksk}_{\text{LWE} \rightarrow \text{LWE}}$. In

MPC, the key generation is still easily achieved by establishing and opening the random seed, after which the parties can locally expand the seed and consider the resulting values as public information. The same random seed can be used for all different required key switching keys, as long as distinct randomness is extracted from it for each key.

Similarly, when parties need to send many inputs to the server, it may be beneficial to apply the same technique to the LWE ciphertexts encrypted under s_i , reducing the cost of sending t inputs from $t \cdot (n + 1) \cdot \log q$ to only $t \cdot \log q + \lambda$ bits. In either of these cases, the impact on the computational cost would be minor in comparison to the rest of the scheme. Where needed for performance, the expansion can still be performed a single time up-front and stored in memory, keeping the improvement of this compression confined to the amount of data communicated over the network.

5 Implementation

In order to show the practicality and correctness of our construction, we implement the multiparty version of FINAL as described in Section 4. Note that this is a prototype implementation and does not realize the full capabilities of FINAL (e.g. our bootstrapping takes 88.6ms while the original implementation manages to do this in 48ms). Using similar adaptations any alternative implementation can be modified to provide multiparty functionality.

As we treat the MPC preprocessing and key generation as a black box, our code implements the ideal functionality directly as a single program, and does not try to emulate network communication or any specific MPC protocols. Our benchmarks use the original parameter from [BIP⁺22] with minor adjustments to the noise parameters in the LWE scheme, i.e. $\sigma = 3.5$ instead of $\sigma = 4.39$. This change does not affect the security level and, according to [APS15], still provides a security level of $> 2^{110}$ as guaranteed by FINAL. Moreover, this guarantees that the decryption error of MPFHE FINAL is $< 2^{-75}$ in the NGS scheme and 2^{-40} in the LWE scheme. Note that these changes do not obstruct the improvements to FINAL as suggested in [JMPP24].

Due to the application of the framework, the only limitation that the number of parties can impose on the scalability of the scheme is the number and size of the key switching keys. As can be seen in Table 2, we note that the storage of the key material is not restrictive requiring at most 22.7 GB when the protocol is executed by 512 parties. While this is not part of our implementation, we recall that the memory footprint of the key switching keys could be significantly reduced by generating random seeds in the key generation algorithm that can be expanded into full LWE samples. Additionally, the work and time required for the multiparty key generation naturally scales linearly with the number of parties.

Concretely, our implementation generates the communal LWE and NGS keys $\mathbf{s}$ and f (but does not output these), and the associated evaluation keys for the regular FINAL protocol. Additionally, it generates k input keys $\mathbf{s}_i$, one per party, along with the key switching keys $\text{ksk}_{\mathbf{s}_i \rightarrow \mathbf{s}}$ and a (NGS to LWE) key switching key $\text{ksk}_{f \rightarrow \mathbf{s}_1 \dots \mathbf{s}_k}$ for the output, which will both be part of the evaluation keys for the server. Evidently, each party receives their personal key $\mathbf{s}_i$. Inputs are provided by the parties as LWE encryptions under their personal keys,⁶ which the server can transform into encryptions under $\mathbf{s}$ through $\text{ksk}_{\mathbf{s}_i \rightarrow \mathbf{s}}$ followed by a single bootstrap operation for noise management. From there, any choice of logic gates can be executed by following the single-key scheme. Output towards the parties

⁶ Observe that here too, network communication could be reduced (in the case of many inputs) by compressing the ciphertexts into a single random seed with one extra field element per input.

is performed by executing a final bootstrap operation on the relevant bits, where the final key switching step makes use of $\text{ksk}_{f \rightarrow \mathbf{s}_{1 \dots k}}$ instead of the usual $\text{ksk}_{f \rightarrow \mathbf{s}}$. We take this approach, rather than performing a key switch from $\mathbf{s}$ so $\mathbf{s}_{1 \dots k}$, to limit the final output noise and prevent decryption failures. Depending on the specific application,⁷ this need not be a separate bootstrap operation, but could this be integrated into the bootstrap associated with the operation the computed the output bit. To decrypt these final MKLWE ciphertexts, we simply assume all parties are present and can decrypt in a round-robin fashion.

Performance

The benchmarks were performed single-threaded, on a laptop with 64 GB of RAM and an Intel Core Ultra 7 155H CPU. As we do not aim for our implementation to be specifically optimized for running time, we focus in our presentation of these benchmarks on the effect of scaling to higher numbers of parties, and as such do not aim for minute control on timing fluctuations and the potential impact of other running processes on the machine. In particular, we aim to show that key generation, as well as output and decryption operations scale linearly with the number of parties, while everything else performs at a constant cost similar to what we observe in our single-key FINAL figures.

In table 4, we first summarize a few running times of our implementation of the single-key FINAL scheme. Then, we present our measurements of multiparty key generation, the final key switching output, and multi-key decryption for 2, 4, 8, 16 and 32 parties in section 5. The implementation is not limited to the number of parties being a power of two, and can scale to arbitrarily many parties without any increase in the parameters, but these choices should give a clear overview of the scaling behavior for our scheme.

As we expect, we observe that input and gate evaluation timings are in line with a single-key bootstrap, with some minor overhead to perform the key switch or addition of the two ciphertexts respectively. For key generation, output and decryption, we observe a linear scaling in the number of parties, matching the expected extra costs of needing MKLWE with one key per party and the generation of key switching keys per party.

Protocol k	Key Size in GB								
	2	4	8	16	32	64	128	256	512
PvLZ [PvLZ24]	.32	4.5	119	5337	-	-	-	-	-
PR [PR23, Table 3]	.42	.63	1.1	2.2	-	-	-	-	-
KMS [KMS24, Table 3]	.215	0.285	.25	.285	.321	-	-	-	-
AKÖ [AKÖ23, Table 3]	.08	.24	.66	.93	2.0	4.1	9.1	37	80
This work	.13	.22	.39	.75	1.45	2.87	5.7	11.4	22.7

Table 2: A comparison of total key material in GB where k is the number of participating parties, [PR23] counts only the size of bsk and not ksk.

In table 2 and table 3, we compare the size of our required key material and the time necessary to perform a single bootstrap to prior work. One of the compared works, [PvLZ24], is a multi-key scheme, so the comparison is naturally favorable towards our scheme, but the other schemes

⁷ For instance, when output bits cannot be used as inputs to later boolean gate.

Protocol	Bootstrap time in ms									
k	2	4	8	16	32	64	128	256	512	
PvLZ [PvLZ24]	No Implementation Data available									
PR [PR23, Table 2]	200	330	460	900	-	-	-	-	-	
KMS [KMS24, Table 3]	240	880	2230	5650	13940	-	-	-	-	
AKÖ [AKÖ23, Table 3]	190	560	1200	1800	4300	8600	18000	-	-	
This work	88.6 (Ours) / 48 (Original, [BIP ⁺ 22])									

Table 3: A comparison of time required to perform a bootstrapping operation, where k is the number of participating parties.

we compare to are for the multiparty setting.⁸ We also again remark that our implementation is less performant than the original implementation of the FINAL scheme, but that this is purely an artifact of the implementation itself and replacing our inner single-key FHE implementation with the more performant version would be possible without any further changes to the key size, parameter choices, or security analysis.

Key generation in MPC

A wide variety of different MPC protocols exist that may be useful in a variety of situations, with support for different types of access structures and many tradeoffs that depend on where the protocol is applied or deployed. As our framework is agnostic to the specific flavor of MPC being employed, and we want to enable a wide variety of potential choices, including e.g. choosing between an honest majority assumption or a full-threshold collusion assumption, we do not provide an implementation of a full MPC protocol for our key generation. Instead, we provide an estimation of the required communication and round cost of such a protocol, in the linear secret sharing-based MPC paradigm. We identify four different components of the key generation that can be analyzed independently. By inspection, each of the components can be seen to require only a constant amount of rounds, and hence so does the overall key generation. If so desired, the components can be interleaved to even further reduce the total number of required rounds. Communication overhead that grows quadratically in the number of parties can be avoided, where needed, by applying the king paradigm from [DN07].

For the base MPC protocol, we assume the following functionalities and building blocks to be present, and we do not make any assumptions on the access structure underlying the secret sharing. These functionalities are commonly found in existing protocol, and can frequently be built from more fundamental building blocks. Where relevant, we suggest such constructions. Observe that additionally, when dealing with an honest majority or $\mathcal{Q}_2$ access structure, some of the operations may be optimized further, for instance by letting the result of a multiplication be opened directly during the execution of the multiplication.

Open A share can be opened $x \leftarrow \text{Open}([x])$ in a constant number of rounds and with a constant number of field elements communicated per party.

Multiplication As a basic operation, the multiplication of two shares $[a \cdot b] \leftarrow \text{Mult}([a], [b])$ can be performed in a constant number of rounds and with a constant number of field elements per party.

⁸ Notably, for [AKÖ23], we compare against the *static* scheme.

Pseudo-random secret sharing Parties can establish, without communication, a sharing of a random, unknown value, uniformly over the field: $[x] \leftarrow \text{PRSS}()$. This may, for instance, be achieved with techniques from [CDI05]. Observe that for an additive secret sharing scheme, it suffices for every party to independently sample their own share.

Pseudo-random bit sharing Parties can establish a sharing of a random bit (0 or 1 in the field) at the cost of a squaring and an opening: $[b] \leftarrow \text{PRBS}()$. One way to do this stems from [DKL⁺13]: obtain $[r] \leftarrow \text{PRSS}()$, then reveal $r^2 \leftarrow \text{Open}(\text{Mult}([r], [r]))$. Now the random bit sharing can be computed as $[b] = \frac{1 + (\sqrt{r^2})^{-1} \cdot [r]}{2}$.

Multiplicative inverse It is possible to find the multiplicative inverse of a shared value at the cost of a multiplication and an opening. One way to achieve this is to obtain a mask $a \leftarrow \text{PRSS}()$, then reveal $\alpha \leftarrow \text{Open}(\text{Mult}([a], [x]))$ and obtain $[x^{-1}] = \alpha^{-1}[a]$. For our purposes, any inverse of zero that shows up in this procedure can be ignored, as they will lead to retrying the computation with fresh randomness.

From these primitives, we can build up the required four components of the key generation. For reasons that will become clear below, we change the **LWE** modulus $q = 133121 = 2^{11} \cdot 5 \cdot 13 + 1$, which does not notably change any correctness, security or performance characteristics of **FINAL**. Additionally, we also will rely on **LWE** noise following the centered binomial distribution $e = \sum_{i=1}^{38} b_i - b'_i$ with b_i and b'_i uniformly random bits. This can be confirmed to have almost the same standard deviation as the discrete Gaussian distribution used by **FINAL** for the noise growth analysis, and to not lose any security as computed by the lattice estimator [APS15].

Common **LWE key s** The common **LWE** key s consists of n random bits, so the parties can invoke n parallel instances of **PRBS**.

Bootstrapping key bsk The bootstrapping key consists of encryptions of s under an **NTRU** key. Generating the **NTRU** key polynomial f can be done in $2 \cdot N$ invocations of **PRBS**, as the distribution of the coefficients can be achieved by computing $\text{PRBS}() - \text{PRBS}()$. To compute the polynomial f^{-1} , we can make use of our change in modulus, which now allows for the easy computation of an **NTT** of degree N . The **NTT** computation is linear, so this step can happen entirely locally. In the **NTT** representation, we can then simply invert f componentwise. If any **NTT** component is zero, or the mask used for inversion is zero, a new candidate f is sampled. All bsk_i can now be computed as $\frac{\mathbf{g}}{f} + \mathbf{g}s_i$, requiring $2 \cdot \ell \cdot N \cdot n$ total random bits to sample $\mathbf{g}$ similar to f and $N \cdot n$ multiplications (in the **NTT** domain).

Output key switching key $\text{ksk}_{\text{NGS} \rightarrow \text{MKLWE}}$ The final key switch of the bootstrapping, and from our optimization, the final key switch to the output **MKLWE** key encrypts the coefficients of f respectively under the common **LWE** key s , or jointly under all individual keys. We write $\hat{\mathbf{f}} = (f_0, -f_{N-1}, -f_{N-2}, \dots, -f_1)$, and $(\text{ksk}_{\text{NGS} \rightarrow \text{MKLWE}})_j = \sum_i A_{j,i} \cdot s_i + \mathbf{e}_j + \mathbf{g}\hat{\mathbf{f}}_j$. All matrices A_i can be publicly computed from shared **PRNG** seeds, and hence have no communication cost. The matrix-vector multiplication and multiplication by the gadget vector are linear, and can be computed locally. This only leaves $\mathbf{e}_j$ to be sampled from the centered binomial distribution, at a total cost of $2 \cdot 38 \cdot \ell \cdot N$ calls to **PRBS**.

Input key switching keys $\text{ksk}_{\text{LWE} \rightarrow \text{LWE}}$ For the input key switching keys, transforming ciphertexts from the individual key s_i of a party to the common key s , we need to compute $(\text{ksk}_{\text{LWE} \rightarrow \text{LWE}})_j = \langle \mathbf{a}_j, s \rangle + \Delta(s_i)_j + e_j$. Since the privacy of only s_i is at stake, the e_j may be directly secret shared by the owner of s_i , for a total cost of n sharings per party.

In total, this results in an MPC protocol for the key generation that runs in a constant number of rounds, and requires a total of $((2 \cdot 2 \cdot 38 + 2 \cdot n) \cdot \ell + 2) \cdot N + n = \mathcal{O}(n \cdot \ell \cdot N)$ random bit generations, $N \cdot n$ multiplications, N inversions, and $2 \cdot n$ input sharings per party. For the parameters of our protocol, this amounts to fewer than ten million multiplications and openings.

LWE.Enc	LWE.Dec	Gate evaluation (Bootstrap)	Key gen	$\text{KS}_{\text{LWE} \rightarrow \text{LWE}}$
178 μs	1.39 μs	88.65 ms	3.43 s	2.71 ms

Table 4: An overview of our running times for basic operations in the single-key FINAL and LWE schemes.

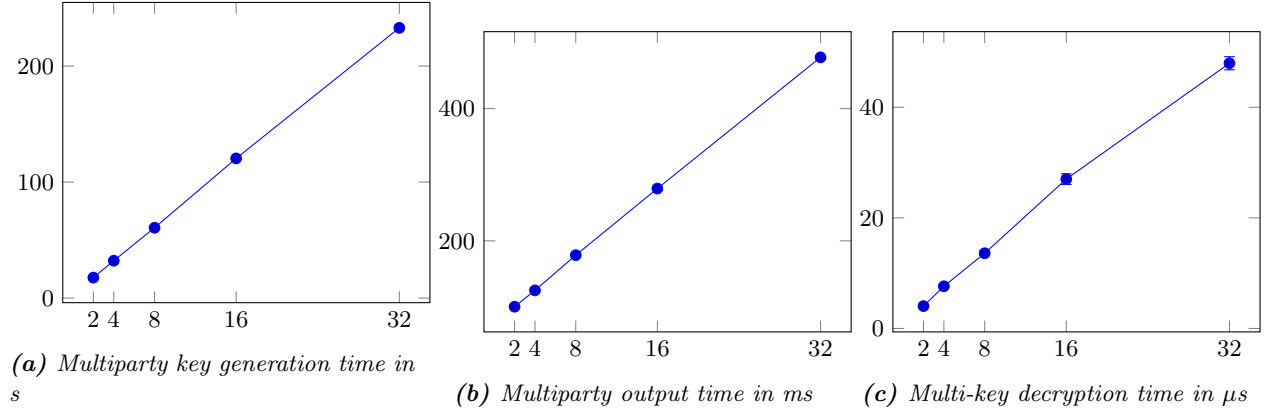


Fig. 11: Experimental verification that the running time (y -axis) for multiparty key generation, key switching from NGS to MKLWE, and MKLWE decryption grows linearly in the number of parties (x -axis).

Acknowledgements

This work was supported by the Flemish Government through the Cyber Security Research Program with grant number VOEWICS02 and VR20192203. This work has been supported in part by the FWO under an Odysseus project GOH9718N.

Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the Cyber Security Research Flanders or the FWO.

References

- AJJM20. Prabhanjan Ananth, Abhishek Jain, Zhengzhong Jin, and Giulio Malavolta. Multi-key fully-homomorphic encryption in the plain model. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part I*, volume 12550 of *LNCS*, pages 28–57. Springer, Cham, November 2020.
- AKÖ23. Yavuz Akin, Jakub Klemsa, and Melek Önen. A practical TFHE-based multi-key homomorphic encryption with linear complexity and low noise growth. In Gene Tsudik, Mauro Conti, Kaitai Liang, and Georgios Smaragdakis, editors, *ESORICS 2023, Part I*, volume 14344 of *LNCS*, pages 3–23. Springer, Cham, September 2023.

- APS15. Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, 9(3):169–203, 2015.
- BCG⁺17. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, and Michele Orrù. Homomorphic secret sharing: Optimizations and applications. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 2105–2122. ACM Press, October / November 2017.
- BGV12. Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In Shafi Goldwasser, editor, *ITCS 2012*, pages 309–325. ACM, January 2012.
- BGW88. Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In *20th ACM STOC*, pages 1–10. ACM Press, May 1988.
- BIP⁺22. Charlotte Bonte, Ilia Iliashenko, Jeongeun Park, Hilder V. L. Pereira, and Nigel P. Smart. FINAL: Faster FHE instantiated with NTRU and LWE. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part II*, volume 13792 of *LNCS*, pages 188–215. Springer, Cham, December 2022.
- CDI05. Ronald Cramer, Ivan Damgård, and Yuval Ishai. Share conversion, pseudorandom secret-sharing and applications to secure computation. In Joe Kilian, editor, *TCC 2005*, volume 3378 of *LNCS*, pages 342–362. Springer, Berlin, Heidelberg, February 2005.
- CDKS19. Hao Chen, Wei Dai, Miran Kim, and Yongsoo Song. Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 395–412. ACM Press, November 2019.
- CDKS21. Hao Chen, Wei Dai, Miran Kim, and Yongsoo Song. Efficient homomorphic conversion between (ring) LWE ciphertexts. In Kazue Sako and Nils Ole Tippenhauer, editors, *ACNS 2021, Part I*, volume 12726 of *LNCS*, pages 460–479. Springer, Cham, June 2021.
- CGGI16. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part I*, volume 10031 of *LNCS*, pages 3–33. Springer, Berlin, Heidelberg, December 2016.
- CGGI20. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, January 2020.
- CL25. Yijia Chang and Songze Li. Arbitrary-threshold fully homomorphic encryption with lower complexity. In *Proceedings of the 34th USENIX Conference on Security Symposium*, SEC '25, USA, 2025. USENIX Association.
- CNT12. Jean-Sébastien Coron, David Naccache, and Mehdi Tibouchi. Public key compression and modulus switching for fully homomorphic encryption over the integers. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 446–464. Springer, Berlin, Heidelberg, April 2012.
- COS⁺22. Ilaria Chillotti, Emmanuela Orsini, Peter Scholl, Nigel P. Smart, and Barry van Leeuwen. Scooby: Improved multi-party homomorphic secret sharing based on FHE. In Clemente Galdi and Stanislaw Jarecki, editors, *SCN 22*, volume 13409 of *LNCS*, pages 540–563. Springer, Cham, September 2022.
- DKL⁺13. Ivan Damgård, Marcel Keller, Enrique Larraia, Valerio Pastro, Peter Scholl, and Nigel P. Smart. Practical covertly secure MPC for dishonest majority - or: Breaking the SPDZ limits. In Jason Crampton, Sushil Jajodia, and Keith Mayes, editors, *ESORICS 2013*, volume 8134 of *LNCS*, pages 1–18. Springer, Berlin, Heidelberg, September 2013.
- DM15. Léo Ducas and Daniele Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 617–640. Springer, Berlin, Heidelberg, April 2015.
- DN07. Ivan Damgård and Jesper Buus Nielsen. Scalable and unconditionally secure multiparty computation. In Alfred Menezes, editor, *CRYPTO 2007*, volume 4622 of *LNCS*, pages 572–590. Springer, Berlin, Heidelberg, August 2007.
- DPSZ12. Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 643–662. Springer, Berlin, Heidelberg, August 2012.
- FV12. Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144, 2012.
- GMW91. Oded Goldreich, Silvio Micali, and Avi Wigderson. Proofs that yield nothing but their validity or all languages in NP have zero-knowledge proof systems. *Journal of the ACM*, 38(3):691–729, July 1991.

- GPvL23. Antonio Guimarães, Hilder V. L. Pereira, and Barry van Leeuwen. Amortized bootstrapping revisited: Simpler, asymptotically-faster, implemented. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part VI*, volume 14443 of *LNCS*, pages 3–35. Springer, Singapore, December 2023.
- JMPP24. Robin Jadoul, Axel Mertens, Jeongeun Park, and Hilder V. L. Pereira. NTRU-based FHE for larger key and message space. In Tianqing Zhu and Yannan Li, editors, *ACISP 24, Part I*, volume 14895 of *LNCS*, pages 141–160. Springer, Singapore, July 2024.
- KMS24. Hyesun Kwak, Seonhong Min, and Yongsoo Song. Towards practical multi-key TFHE: Parallelizable, key-compatible, quasi-linear complexity. In Qiang Tang and Vanessa Teague, editors, *PKC 2024, Part II*, volume 14604 of *LNCS*, pages 354–385. Springer, Cham, April 2024.
- KTSJ24. Nirajan Koirala, Jonathan Takeshita, Jeremy Stevens, and Taeho Jung. Summation-based private segmented membership test from threshold-fully homomorphic encryption. *PoPETs*, 2024(4):209–225, October 2024.
- Lin20. Yehuda Lindell. Secure multiparty computation. *Commun. ACM*, 64(1):86–96, December 2020.
- LTV12. Adriana López-Alt, Eran Tromer, and Vinod Vaikuntanathan. On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In Howard J. Karloff and Toniann Pitassi, editors, *44th ACM STOC*, pages 1219–1234. ACM Press, May 2012.
- MS18. Daniele Micciancio and Jessica Sorrell. Ring packing and amortized FHEW bootstrapping. In Ioannis Chatzigiannakis, Christos Kaklamani, Daniel Marx, and Donald Sannella, editors, *ICALP 2018*, volume 107 of *LIPICs*, pages 100:1–100:14. Schloss Dagstuhl, July 2018.
- MTBH21. Christian Mouchet, Juan Ramón Troncoso-Pastoriza, Jean-Philippe Bossuat, and Jean-Pierre Hubaux. Multiparty homomorphic encryption from ring-learning-with-errors. *PoPETs*, 2021(4):291–311, October 2021.
- MW16. Pratyay Mukherjee and Daniel Wichs. Two round multiparty computation via multi-key FHE. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 735–763. Springer, Berlin, Heidelberg, May 2016.
- MWA⁺23. Yiping Ma, Jess Woods, Sebastian Angel, Antigoni Polychroniadou, and Tal Rabin. Flamingo: Multi-round single-server secure aggregation with applications to private federated learning. In *2023 IEEE Symposium on Security and Privacy*, pages 477–496. IEEE Computer Society Press, May 2023.
- OT25. Hiroki Okada and Tsuyoshi Takagi. Low communication threshold FHE from standard (module-)LWE. Cryptology ePrint Archive, Paper 2025/409, 2025.
- Par21. Jeongeun Park. Homomorphic encryption for multiple users with less communications. Cryptology ePrint Archive, Report 2021/1085, 2021.
- PR23. Jeongeun Park and Sergi Rovira. Efficient TFHE bootstrapping in the multiparty setting. *IEEE Access*, 11:118625–118638, 2023.
- PS24. Alain Passelègue and Damien Stehlé. Low communication threshold fully homomorphic encryption. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part I*, volume 15484 of *LNCS*, pages 297–329. Springer, Singapore, December 2024.
- PvLZ24. Jeongeun Park, Barry van Leeuwen, and Oliver Zajonc. FINALLY: A multi-key FHE scheme based on NTRU and LWE. *IACR Communications in Cryptology*, 1(3), 2024.
- PVZ24. Jeongeun Park, Barry Van Leeuwen, and Oliver Zajonc. FINALLY: A multi-key FHE scheme based on NTRU and LWE. *CiC*, 1(3):15, 2024.